第十章作业 参考答案

> **说明**：各题之后引用块中列出的力扣 / OpenJudge 编程题，只是与该题知识点对应的课外练习，仅供参考，不属于本次作业要求，无需提交。

1 二分检索的判定树与平均检索长度

> 参考练习（不要求提交）：力扣 704. 二分查找；力扣 35. 搜索插入位置（检索失败时返回目标所在的间隙）。
>
> **E704. 二分查找**
>
> https://leetcode.cn/problems/binary-search/
>
> **E35. 搜索插入位置**
>
> https://leetcode.cn/problems/search-insert-position/

(1) 伪代码与判定树

```
BinarySearch(a, n, key):
    left = 0
    right = n - 1

    while left <= right:
        mid = (left + right) // 2        // 向下取整
        if a[mid] == key:
            return mid
        if key < a[mid]:
            right = mid - 1
        else:
            left = mid + 1

    return -1
```

判定树如下（每个结点的左子树对应更小的关键字，右子树对应更大的关键字）：

```
                    40
                /       \
              19          61
             /   \       /   \
            5     26    47    75
             \      \     \   /  \
             12     33    54 68  82
```

根 40 是 `a[5]`（$\lfloor (0+11)/2 \rfloor = 5$），左子区间 `[0, 4]` 的中点是 `a[2] = 19`，右子区间 `[6, 11]`

的中点是 `a[8] = 61`，依此类推。

(2) 两次具体检索

检索 75：

步骤	left	right	mid	a[mid]	处理
1	0	11	5	40	75 > 40，查右半区
2	6	11	8	61	75 > 61，查右半区
3	9	11	10	75	找到

比较序列为 `40 → 61 → 75`，共 **3 次**，返回下标 **10**。

检索 50：

步骤	left	right	mid	a[mid]	处理
1	0	11	5	40	50 > 40，查右半区
2	6	11	8	61	50 < 61，查左半区
3	6	7	6	47	50 > 47，查右半区
4	7	7	7	54	50 < 54，right 变为 6

此时 `left = 7 > right = 6`，区间为空。比较序列为 `40 → 61 → 47 → 54`，共 **4 次**，检索失败，返回 **-1**。

(3) 成功平均检索长度

检索某关键字的比较次数等于它在判定树中的层数：

比较次数	对应关键字	个数
1	40	1
2	19, 61	2
3	5, 26, 47, 75	4
4	12, 33, 54, 68, 82	5

$$ASL_{\text{成功}} = \frac{1\times 1 + 2\times 2 + 3\times 4 + 4\times 5}{12} = \frac{37}{12} \approx 3.08.$$

(4) 失败平均检索长度

检索失败的比较次数等于目标所落间隙对应的外部结点的父结点层数：

不存在的目标所处间隙	比较次数
小于 5	3
5 与 12 之间	4
12 与 19 之间	4
19 与 26 之间	3

不存在的目标所处间隙	比较次数
26 与 33 之间	4
33 与 40 之间	4
40 与 47 之间	3
47 与 54 之间	4
54 与 61 之间	4
61 与 68 之间	4
68 与 75 之间	4
75 与 82 之间	4
大于 82	4

共 3 个间隙需要 3 次比较、10 个间隙需要 4 次比较：

$$ASL_{\text{失败}} = \frac{3 \times 3 + 10 \times 4}{13} = \frac{49}{13} \approx 3.77.$$

2 散列函数与冲突处理

> 参考练习（不要求提交）：力扣 705. 设计哈希集合（自己实现链地址法或开地址法）；力扣 1. 两数之和（用散列表把检索降到期望$O(1)$）。
>
> **E705. 设计哈希集合**
>
> https://leetcode.cn/problems/design-hashset/
>
> **E1. 两数之和**
>
> https://leetcode.cn/problems/two-sum/

(1) 三种散列函数

- 除余法：$1234 = 97 \times 12 + 70$，$h(1234) = 70$。
- 平方取中法：$1234^2 = 1522756$，补足 8 位为 `01522756`，从左到右第 4、5 位是 2、2，散列地址为 22。
- 折叠法：$(12 + 34) \bmod 100 = 46$。

(2)同义词组与两种散列表

各关键字的初始散列地址：

关键字	22	33	44	55	67	78	89
k mod 11	0	0	0	0	1	1	1

两个同义词组为 `{22, 33, 44, 55}` 和 `{67, 78, 89}`。

链地址法，在相应链表尾部插入：

```
0: 22 → 33 → 44 → 55
1: 67 → 78 → 89
2 至 10: 空链表
```

线性探测法的插入过程：

关键字	依次探测的下标	最终下标	探测次数
22	0	0	1
33	0, 1	1	2
44	0, 1, 2	2	3
55	0, 1, 2, 3	3	4
67	1, 2, 3, 4	4	4
78	1, 2, 3, 4, 5	5	5
89	1, 2, 3, 4, 5, 6	6	6

最终表为：

下标	0	1	2	3	4	5	6	7	8	9	10
内容	22	33	44	55	67	78	89	空	空	空	空

(3) 成功平均检索长度

链地址法从链表表头开始比较，七个关键字所需比较次数分别为 `1, 2, 3, 4, 1, 2, 3`：

$$ASL_{\text{链地址，成功}} = \frac{1+2+3+4+1+2+3}{7} = \frac{16}{7} \approx 2.29.$$

线性探测中没有删除或移动已有记录，成功检索访问的槽数与插入时相同：

$$ASL_{\text{线性，成功}} = \frac{1+2+3+4+4+5+6}{7} = \frac{25}{7} \approx 3.57.$$

(4) 线性探测的失败平均检索长度

查找不存在的关键字要探测到第一个空槽才停止，并计入该空槽：

初始地址	0	1	2	3	4	5	6	7	8	9	10
失败探测次数	8	7	6	5	4	3	2	1	1	1	1

例如从地址 0 出发要访问 0 至 7 共 8 个槽；从地址 7 出发只访问 7 就知道不存在。

$$ASL_{\text{线性，失败}} = \frac{8+7+6+5+4+3+2+1+1+1+1}{11} = \frac{39}{11} \approx 3.55.$$

(5) 同义词、冲突与聚集

- **同义词**：不同关键字经同一散列函数得到相同的初始地址，如 22 与 33 的初始地址都是 0。
- **冲突**：插入或检索时，要访问的槽已被其他关键字占用。同义词必然冲突；处理冲突时后续候选槽也可能被占，需要继续探测。
- **聚集**：线性探测把冲突的记录放到后继槽，形成连续占用区；之后初始地址落在该区内部或紧邻处的关键字都要沿着它探测到末端，又把这个区延长。本题 0 至 6 就形成了长度为 7 的连续区，使 67、78、89 虽然和 22 等并非同义词，也各自多探测了 3 次。

初始散列地址不同的关键字也可能访问同一个槽：33 的初始地址为 0，因 0 被占而存到 1；67 的初始地址恰为 1，因此也要先访问槽 1。冲突处理后的实际存储位置不一定等于初始散列地址，这正是聚集的来源。

3 双散列中的查找、插入与删除

> 参考练习（不要求提交）：力扣 706. 设计哈希映射（带删除的散列表）；力扣 146. LRU 缓存（散列表与双向链表配合）。
>
> **E706. 设计哈希映射**
>
> https://leetcode.cn/problems/design-hashmap/
>
> **M146.LRU 缓存**
>
> https://leetcode.cn/problems/lru-cache/

(1) 初始构表与成功 ASL

关键字 k	h1(k)	d(k)	插入时依次探测的下标	最终下标	探测次数
22	0	3	0	0	1
33	0	4	0, 4	4	2
44	0	5	0, 5	5	2
55	0	6	0, 6	6	2
67	1	8	1	1	1
78	1	9	1, 10	10	2
89	1	10	1, 0, 10, 9	9	4

例如 89：$h_1 = 1$、$d = 10$，依次探测1、$(1 + 10) \bmod 11 = 0$、$(1 + 20) \bmod 11 = 10$、$(1 + 30) \bmod 11 = 9$。最终表为：

下标	0	1	2	3	4	5	6	7	8	9	10
内容	22	67	空	空	33	44	55	空	空	89	78

没有删除时每个关键字成功检索的路径与插入时一致：

$$ASL_{\text{成功}} = \frac{1 + 2 + 2 + 2 + 1 + 2 + 4}{7} = \frac{14}{7} = 2.$$

同一组关键字在线性探测下的成功 ASL 是$25/7$，双散列用与关键字相关的步长把同义词分散开，聚集明显减轻。

(2) 按顺序执行四次操作

删除 22。$h_1(22) = 0$，在槽 0 找到 22，探测 **1 次**。把槽 0 的状态改为 `DELETED`，其他槽不变。

查找 89。$h_1(89) = 1$、$d(89) = 10$，路径为

```
1 (67) → 0 (DELETED) → 10 (78) → 9 (89)
```

在槽 9 找到，探测 **4 次**，表不变。

插入 89。仍沿 `1 → 0 → 10 → 9` 探测，共 **4 次**。经过槽 0 时记住这个第一个 DELETED 位置但继续探测，在槽 9 发现 89 已存在，不插入；槽 0 仍为 DELETED。

插入 199。$h_1(199) = 199 \bmod 11 = 1$，$d(199) = 1 + (199 \bmod 10) = 10$，路径为

```
1 (67) → 0 (DELETED) → 10 (78) → 9 (89) → 8 (EMPTY)
```

共探测 **5 次**。遇到槽 8 的 EMPTY 后可确认 199 不在表中；本次探测遇到的第一个 DELETED 槽是 0，因此把 199 写入槽 0 并改为 OCCUPIED，槽 8 仍为 EMPTY。最终表为：

下标	0	1	2	3	4	5	6	7	8	9	10
内容	199	67	空	空	33	44	55	空	空	89	78

(3) 两种错误做法

删除时直接置空会使查找过早停止。若把槽 0 改为 EMPTY，查找 89 时依次访问 1、0，在槽 0 遇到 EMPTY 就报告不存在，而 89 实际仍在槽 9。DELETED 标记的含义是" 这个槽现在没有记录，但经过它的探测链可能还要继续"。

插入时立即复用删除槽会产生重复记录。插入 89 时第一个 DELETED 槽是 0，但槽 9 已有 89；若直接写入槽 0，表中就出现两个 89，违反关键字不重复的要求，之后的删除也只能删掉其中一个。正确做法是先记住删除位置，继续探测直到遇到 EMPTY 或找到同值记录，再决定是否复用。

(4) 伪代码、全表遍历与复杂度

用 `state[0..10]` 保存槽状态、`value[0..10]` 保存关键字,初始状态全为 EMPTY。只有状态为 OCCUPIED 时 value 才参与比较。

```
Search(key):
    start = key mod 11
    step  = 1 + key mod 10
    for i = 0 to 10:
        pos = (start + i * step) mod 11
        if state[pos] == EMPTY:
            return -1                       // 探测链到头，不存在
        if state[pos] == OCCUPIED and value[pos] == key:
            return pos
        // DELETED，或 OCCUPIED 但关键字不同：继续探测
    return -1
```

```
Insert(key):
    start = key mod 11
    step  = 1 + key mod 10
    firstDeleted = -1
    for i = 0 to 10:
        pos = (start + i * step) mod 11
        if state[pos] == OCCUPIED:
            if value[pos] == key:
                return " 已存在，不重复插入"
        else if state[pos] == DELETED:
            if firstDeleted == -1:
```

```
                firstDeleted = pos          // 记住，但继续探测
        else:                               // EMPTY：已确认关键字不存在
            if firstDeleted != -1:
                pos = firstDeleted
            value[pos] = key
            state[pos] = OCCUPIED
            return " 插入成功"
    // 11 个槽都已检查，没有相同关键字，也没有 EMPTY
    if firstDeleted != -1:
        value[firstDeleted] = key
        state[firstDeleted] = OCCUPIED
        return " 插入成功"
    return " 表满，插入失败"
```

```
Delete(key):
    pos = Search(key)
    if pos == -1:
        return " 不存在，删除失败"
    state[pos] = DELETED                    // 保留删除标记，不置 EMPTY
    return " 删除成功"
```

为什么步长能遍历全表。11 是素数，步长d 在 1 至 10 之间，故$\gcd(d, 11) = 1$。若前 11 次探测中有两个地址相同，设序号为$0 \le i < j \le 10$，则$(j-i)d \equiv 0 \pmod{11}$；由$d$ 与 11 互质得$j - i \equiv 0 \pmod{11}$，但$1 \le j - i \le 10$，矛盾。因此前 11 个地址互不相同，恰好覆盖全表，循环至多 11 次就能给出确定的结论。

复杂度。表长推广为m 且步长与m 互质时，每次查找、插入或删除最坏要探测全部m 个槽，最坏时间$O(m)$，额外空间$O(1)$；负载因子不高时平均探测次数只与负载因子有关，与m 无关（见第 4 题）。

4 几种检索方法的平均检索长度

> 参考练习（不要求提交）：第 (4) 问对应力扣 1. 两数之和、力扣 167. 两数之和 II（输入有序数组）。
>
> **M167. 两数之和 II - 输入有序数组**
>
> https://leetcode.cn/problems/two-sum-ii-input-array-is-sorted/

(1) 按访问频率排序的顺序检索

检索第i 个关键字要比较i 次：

$$ASL = \sum_{i=1}^{n} i \cdot p_i = \sum_{i=1}^{n} \frac{i}{2^i}.$$

记$S = \sum_{i=1}^{n} i/2^i$，则$S - S/2 = \sum_{i=1}^{n} 1/2^i - n/2^{n+1} = 1 - 1/2^n - n/2^{n+1}$，所以

$$ASL = 2 - \frac{n+2}{2^n}.$$

当n 很大时$ASL \to 2$：虽然表长可以任意大，平均只需约 2 次比较。这说明访问概率差异很

大时，按频率降序排列线性表是一种很有效的检索技术。（题目给的$p_n = 1/2^n$ 使概率之和为$1 - 1/2^n$，略小于 1；若把p_n 改为$1/2^{n-1}$ 使之归一，结果为$2 - 1/2^{n-1}$，结论相同。）

(2) 25 个元素的三种检索

顺序检索：$ASL = \dfrac{25+1}{2} = 13$。

二分检索：25 个结点的判定树各层结点数为 1、2、4、8、10（第 5 层未满）：

$$ASL = \frac{1\times 1 + 2\times 2 + 3\times 4 + 4\times 8 + 5\times 10}{25} = \frac{99}{25} = 3.96.$$

分块检索，$b = 5$ 块、每块$s = 5$ 个元素，块内只能顺序检索，平均$(5+1)/2 = 3$ 次：

- 索引表顺序检索：$ASL = \dfrac{b+1}{2} + \dfrac{s+1}{2} = 3 + 3 = 6$；
- 索引表二分检索：5个索引项的判定树层数为 1、2、2、3、3，平均$11/5 = 2.2$ 次，$ASL = 2.2 + 3 = 5.2$。教材的近似公式$\log_2(b+1) + (s-1)/2 = \log_2 6 + 2 \approx 4.58$ 是把索引检索按$\log_2(b+1) - 1$ 估计的结果，量级一致。

三种方法的 ASL 依次为 13、3.96、6（或 5.2）：二分最快但要求有序且不便插删，分块介于两者之间，便于块内插删。

(3) 散列的理论值与实测值

$\alpha = 7/11 \approx 0.636$，$1 - \alpha = 4/11$：

方法	理论值	实测值（第 2 题）
开散列，成功	$1 + \alpha/2 = 29/22 \approx 1.32$	$16/7 \approx 2.29$
线性探测，成功	$\frac{1}{2}\left(1 + \frac{11}{4}\right) = 15/8 = 1.875$	$25/7 \approx 3.57$
线性探测，失败	$\frac{1}{2}\left(1 + \frac{121}{16}\right) = 137/32 \approx 4.28$	$39/11 \approx 3.55$

理论公式假设关键字的初始地址在表中均匀分布（探测序列近似随机），且表长很大。第 2 题的 7 个关键字只落在两个初始地址上：22、33、44、55 都是 11 的倍数，67、78、89 都比 11 的倍数大 1，散列函数$k \bmod 11$ 把它们完全压到地址 0 和 1，是最坏的同义词分布，所以链地址法的链特别长、线性探测形成一整块聚集，成功检索长度约为理论值的两倍。失败检索实测反而略低于理论值，是因为表很小、占用槽集中成一块，初始地址 7 至 10 一步就碰到空槽；在大表中这种偏差会被平均掉。结论是：散列的效率不仅取决于负载因子，还取决于散列函数是否适合实际数据，表长取素数并不能挽救 “关键字本身就有以表长为周期的规律” 这种情况。

(4) 两数之和为 K

排序 + 二分检索，$O(n \log n)$。先把数组排序（$O(n \log n)$），再对每个元素x 在整个有序数组中二分检索$K - x$（允许找到x 自身，对应同一元素使用两次），共n 次检索、每次$O(\log n)$：

```
TwoSumSorted(a, n, K):
    sort a in nondecreasing order
    for i = 0 to n - 1:
        if BinarySearch(a, n, K - a[i]) != -1:
            return true
```

```
    return false
```

排序后也可以用双指针：i 从左端、j 从右端，和小于K 则 i 右移，大于则 j 左移，等于则成功（i == j 时对应同一元素用两次），检索阶段只需$O(n)$，总时间仍由排序决定。

散列表，期望$O(n)$。把所有元素插入散列表，再对每个元素x 查$K-x$ 是否在表中；每次插入和检索平均$O(1)$：

```
TwoSumHash(a, n, K):
    H = empty hash set
    for i = 0 to n - 1:
        insert a[i] into H
    for i = 0 to n - 1:
        if (K - a[i]) in H:
            return true
    return false
```

对 8，4，1，6、$K=10$：查$10-4=6$ 时命中，返回 true。若不允许同一元素用两次，改为边扫描边插入、先查后插即可。散列法以$O(n)$ 额外空间换取线性期望时间，最坏情况（全部冲突）退化为$O(n^2)$；排序法不需要额外的散列结构，最坏时间也有保证。

> 核对说明：第 1、2、3 题取自本次作业的原始题目，原答案经脚本逐题复核无误（二分判定树与 37/12、49/13；三种散列函数；线性探测 25/7、39/11；双散列各探测序列与四次操作）。第 4 题为依据教材 10.1、10.3.5 节及本章习题 1、4、6 新增。